PIPELINE ETL – TRÁFICO AÉREO T100 2023

Guía de Ejecución — Etapa Transform & Load

Atributo	Detalle
Proyecto	Data Warehouse – Análisis de Tráfico Aéreo
Dataset	BTS T100 Segment – All Carriers 2023
Motor BD	SQL Server (T-SQL)
Etapa	Transform (T) + Load (L)
Scripts	create_oltp.sql | transform_load.sql | verify_load.sql
Prerrequisito	Staging cargado: dw_staging_raw con datos de extract.py

1. Arquitectura del Pipeline

El pipeline sigue la arquitectura ETL en tres capas independientes:

Capa	Base de datos	Propósito	Script asociado
Extract (E)	dw_staging_raw	Recibe los CSV crudos sin transformación ni constraints	extract.py
Transform (T)	dw_staging_raw	Limpieza, deduplicación y aplicación de reglas de negocio en memoria SQL	transform_load.sql
Load (L)	dw_oltp	Modelo OLTP normalizado con FK, PK e índices analíticos	transform_load.sql

Idempotencia

Todos los scripts son re-ejecutables. create_oltp.sql elimina y recrea las tablas. transform_load.sql borra el año objetivo antes de recargar.

No se deben ejecutar en producción con datos de períodos anteriores sin revisar la variable @AnioRecarga.

2. Prerrequisitos

2.1 Software requerido

Componente	Versión mínima	Notas
SQL Server	2016 (13.x)	Express admitido, pero sin agente SQL
SSMS o Azure Data Studio	Cualquiera	Para ejecutar los scripts .sql

Componente	Versión mínima	Notas
Python	3.8+	Solo para la etapa Extract (extract.py)
ODBC Driver	17 o 18	Para conexión Python → SQL Server

2.2 Estado esperado antes de ejecutar

- La base de datos dw_staging_raw debe existir y estar poblada.
- Las 21 tablas stg_* deben tener datos (ejecutar extract.py previamente).
- El usuario SQL debe tener permisos CREATE DATABASE, CREATE TABLE, INSERT, DELETE sobre ambas bases.
- Verificar conectividad al servidor con la cadena de conexión del .env.

Verificar staging antes de continuar

Ejecutar en SSMS: SELECT name, SUM(p.rows) FROM sys.tables t JOIN sys.partitions p ON p.object_id=t.object_id WHERE t.name LIKE 'stg_%' GROUP BY name

Todas las tablas stg_* deben mostrar filas > 0. Si alguna está vacía, re-ejecutar extract.py.

3. Estructura de Archivos del Proyecto

```
proyecto/
├── sources/                    ← CSVs originales del BTS
│   ├── t100_carriers_2023.csv
│   └── l_*.csv                 ← lookups
├── .env                        ← cadena de conexión (NO versionar)
├── .env.example
├── requirements.txt
│
├── extract.py                  ← Etapa E: carga staging
│
├── create_oltp.sql             ← Crea BD destino dw_oltp  ← NUEVO
├── transform_load.sql          ← Etapa T+L: limpieza y carga
└── verify_load.sql             ← Validación post-carga    ← NUEVO
```

4. Pasos de Ejecución — Orden Obligatorio

Respetar el orden estrictamente

Los pasos 1-2 son de infraestructura (se ejecutan una sola vez por entorno).

Los pasos 3-4 son el ciclo de carga (se repiten por cada período de datos).

Nunca ejecutar transform_load.sql si create_oltp.sql no se completó sin errores.

PASO 1

Verificar que el Staging está cargado

Abrir SSMS → conectar a la instancia → ejecutar la consulta de verificación de staging del paso 2.2. Confirmar que stg_t100 tiene ~98.622 filas y los lookups tienen datos.

PASO 2

Ejecutar create_oltp.sql — Crear la BD destino

Abrir create_oltp.sql en SSMS. Revisar que no exista ya una base dw_oltp con datos que no deban perderse (el script la recrea desde cero). Ejecutar con F5. Confirmar mensaje final: "✓ Modelo OLTP dw_oltp creado exitosamente."

PASO 3

Ejecutar transform_load.sql — Transformación y Carga

Abrir transform_load.sql en SSMS. Verificar la variable @AnioRecarga (por defecto 2023). El script carga las dimensiones primero y la tabla de hechos al final. Tiempo estimado: 2-10 minutos según hardware. Monitorear los mensajes PRINT en la pestaña Messages.

PASO 4

Ejecutar verify_load.sql — Validación post-carga

Abrir verify_load.sql en SSMS y ejecutar. Revisar que la sección "Violaciones de Reglas de Negocio" muestre 0 en todas las filas. Revisar conteos de la sección "Métricas Generales" para sanity check.

4.1 Cómo ejecutar cada script en SSMS

```
1. Abrir SQL Server Management Studio (SSMS)
2. File → Open → File...  (Ctrl+O)
3. Seleccionar el archivo .sql
4. Verificar que el selector de BD en la barra superior dice
   "master" para create_oltp.sql
   "dw_oltp" para transform_load.sql y verify_load.sql
5. Presionar F5 (o botón Execute) para ejecutar
6. Revisar la pestaña "Messages" al finalizar
```

Consejo: modo SQLCMD para automatización

Si se requiere ejecución desatendida (por ej. en un job del Agente SQL), se puede usar:

sqlcmd -S localhost -d master -i create_oltp.sql

sqlcmd -S localhost -d dw_oltp -i transform_load.sql

sqlcmd -S localhost -d dw_oltp -i verify_load.sql

5. Reglas de Calidad Implementadas

Las siguientes 11 reglas se aplican durante la ejecución de transform_load.sql. Todas están comentadas en el código con la etiqueta R01–R11.

ID	Regla	Acción	Ubicación en el script
R01	AirlineID como clave estable del carrier (no UniqueCarrier)	Ignorar sufijo "(1)" del código IATA	CTE carriers_raw
R02	Normalizar UniqueCarrierName	UPPER + TRIM + quitar Inc./Co./Ltd.	fn_NormalizarNombreCarrier
R03	AirlineID = 0 o NULL	Mapear a carrier "DESCONOCIDO"	INSERT especial + COALESCE en hechos
R04	DepPerformed > DepScheduled	DESCARTE del segmento	WHERE de base_limpia
R05	Passengers > Seats o Seats=0 con pax	DESCARTE del segmento	WHERE de base_limpia

ID	Regla	Acción	Ubicación en el script
R06	Distance ≤ 0	DESCARTE del segmento	WHERE de base_limpia
R07	AirTime ≤ 0 o > 1.200 min	DESCARTE del segmento	WHERE de base_limpia
R08	Freight > Payload o Freight < 0	DESCARTE del segmento	WHERE de base_limpia
R09	AirportSeqID confundido con AirportID	JOIN por SeqID; descartar si SeqID=AirportID	AEROPUERTOS + base_limpia
R10	Filas duplicadas en stg_t100	ROW_NUMBER() PARTITION BY clave natural	CTE base_dedup
R11	Flag Activo de carriers inactivos	Cruzar EndDate con GETDATE()	CTE activo_calc

6. Modelo OLTP Destino (dw_oltp)

El modelo se carga en el siguiente orden de dependencia. Las tablas sin flecha son independientes (sin FK entrantes de otras tablas del modelo).

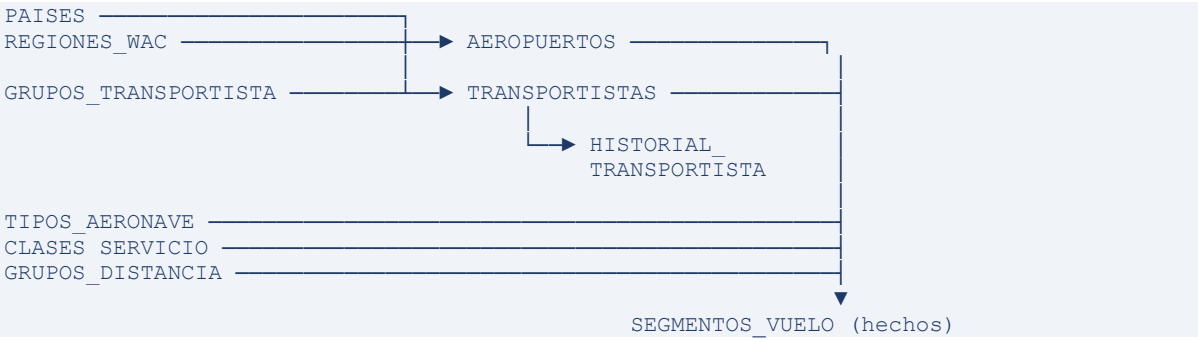

Tabla	Tipo	Filas esperadas (aprox.)	Clave natural
PAISES	Dimensión	~60	CodigoPais
REGIONES_WAC	Dimensión	~350	CodigoWAC
AEROPUERTOS	Dimensión	~800	AirportSeqID
GRUPOS_TRANSPORTISTA	Dimensión	~15	CodigoGrupo
TRANSPORTISTAS	Dimensión	~45	AirlineID
HISTORIAL_TRANSPORTISTA	Histórico	Variable	TransportistaID + FechaCambio
TIPOS_AERONAVE	Dimensión	~100	CodigoGrupo+CodigoTipo+Config
CLASES_SERVICIO	Dimensión	~10	CodigoClase
GRUPOS_DISTANCIA	Dimensión	~12	CodigoGrupo
SEGMENTOS_VUELO	Hechos	~85.000+	SegmentoID (IDENTITY)

7. Resolución de Problemas Frecuentes

Error: "Cannot open database dw_staging_raw"

- Causa: el staging no existe o la cadena de conexión apunta a otra instancia.
- Solución: verificar que extract.py se ejecutó correctamente. Ejecutar: SELECT name FROM sys.databases para listar las bases disponibles.

Error: "The INSERT statement conflicted with the FOREIGN KEY constraint"

- Causa: se intentó cargar SEGMENTOS_VUELO antes de que las dimensiones estuvieran completas.
- Solución: no ejecutar el script parcialmente. Ejecutarlo siempre completo desde el principio.

SEGMENTOS_VUELO tiene muchas menos filas de las esperadas (~85.000)

- Causa normal: las reglas de calidad (R04–R08) descartan filas inválidas. El número exacto depende de los datos inyectados.
- **Diagnóstico:** ejecutar verify_load.sql y revisar la sección "Violaciones de Reglas".
- Si hay 0 violaciones y los conteos son consistentes, la carga es correcta.

Carriers con Activo=0 aparecen en SEGMENTOS_VUELO

- Esto es esperado y correcto. Un carrier puede haber cerrado en 2023 pero tener vuelos en los meses anteriores al cierre dentro del mismo año.
- verify_load.sql muestra estos carriers en la sección 7 para revisión manual.

El script tarda más de 15 minutos

- Verificar que dw_staging_raw no tenga locks activos de otra sesión.
- Ejecutar: SELECT session_id, blocking_session_id, wait_type FROM sys.dm_exec_requests para detectar bloqueos.

8. Recarga Parcial por Período

Para recargar solo un año/mes sin destruir la BD completa:

```
-- 1. Ajustar la variable al inicio de transform_load.sql:
DECLARE @AnioRecarga INT = 2023;   -- cambiar al año deseado

-- 2. El script ejecuta automáticamente:
DELETE FROM dbo.SEGMENTOS_VUELO WHERE Anio = @AnioRecarga;

-- 3. Luego re-inserta los registros limpios de ese año.

-- NOTA: Las dimensiones (AEROPUERTOS, TRANSPORTISTAS, etc.)
-- se actualizan con MERGE (upsert), por lo que son siempre
-- re-ejecutables sin pérdida de datos.
```

9. Glosario

Término	Definición
AirlineID	Identificador numérico estable asignado por el DOT/BTS al carrier. No cambia con fusiones.
UniqueCarrier	Código IATA del carrier. Puede reutilizarse tras una fusión; agregar sufijo "(1)" si hay conflicto.

Término	Definición
AirportSeqID	Versión del AirportID que incluye el período de tiempo. Permite rastrear cambios de atributos del aeropuerto.
CityMarketID	Identificador del mercado urbano. Agrupa aeropuertos de la misma ciudad (EWR/JFK/LGA → NY).
WAC	World Area Code. Clasificación geográfica regional del BTS.
stg_*	Tablas de staging en dw_staging_raw. Contienen datos crudos sin constraints.
MERGE (UPSERT)	Operación SQL que inserta si no existe o actualiza si ya existe, basándose en una clave de join.
Idempotencia	Propiedad de un script que produce el mismo resultado sin importar cuántas veces se ejecute.

Data Engineering — Pipeline ETL Tráfico Aéreo T100 2023